

Quale obiettivo hanno i linguaggi di progr. (PL)?

Dati

Algoritmi

Calcolo da eseguire
sui dati

=> PL = uno strumento formale per descrivere algoritmi applicati ai dati

↳ Programma Realizzazione nel PL dell'algoritmo

↳ realizzazione concreta

↳ realizzazione astratta

=> Algoritmo = seq. di passi primitivi finita descritta attraverso un progr.

in un linguaggio di programmazione

↳ calcola funzioni

=> Programma = rapp. sintattica (rispetta regole sintattiche del PL)
finita di comportamenti; potenzialmente infinit.

↳ funzioni $f: \text{Dom} \rightarrow \text{Cod}$ $\cup \{\uparrow\}$
diverge

↳ la funzione calcolata nell'algoritmo
rappresentato/implementato dal programma

=> Il PL deve considerare strumenti per descrivere metodi di
calcolo

Linguaggio matematico

Notazione rigorosa per rappresentare funzioni:

num. pari = $\{0, 2, 4, 6, \dots\}$

$f(x) = x+2$ no paradigma funzionale, composizione di funzioni
(λ calcolo ML, LISP)

manca la
descrizione del
processo di
calcolo

Linguaggio logico

Notazione rigorosa. Regole-assiomi no calcolo come dimostrazione

manca la descrizione di calcoli potenzialmente infiniti

no paradigma logico (PROLOG)

Linguaggi di programmazione (PL)

Notazione rigorosa (Grammatiche). Regole (semantica delle istruzioni)

Istruzioni che permettono di descrivere i passi di calcolo

Applicazioni: scientifiche → FORTRAN

Applicazioni: economiche → COBOL linguaggio vecchio usato da banche/assicurazioni

Applicazioni: AI → LISP (simbolica?)

Applicazioni: sistemi → C

Applicazioni: Web → Javascript, ..., Java

Aspetti di progettazione

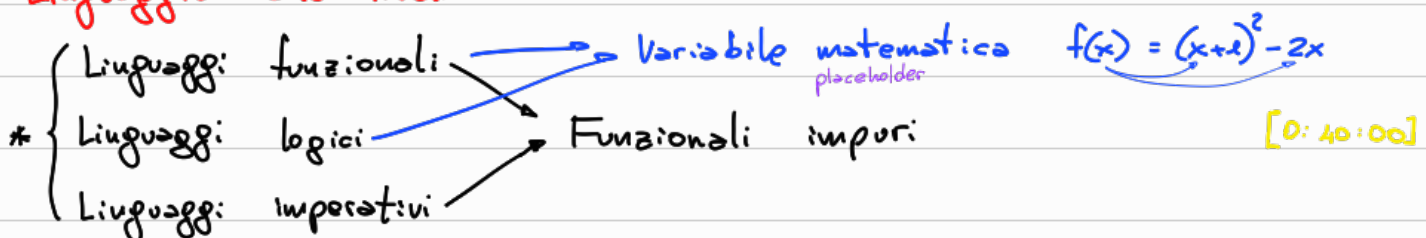
- Readability : sintassi chiara, poca/nessuna ambiguità, facilità di lettura, facile comprensibilità dei programmi
 - Writability : facilità di scrittura, quanto è facile/agevole scrivere un programma nel PL, facilità di analisi del codice
 - Affidabilità : conformità delle specifiche, riduzione di inserimento di errori di programmazione
 - Costo : costi di efficienza
costi di apprendimento
costi di utilizzo nel modo corretto
.....
-

Linguaggio basso livello

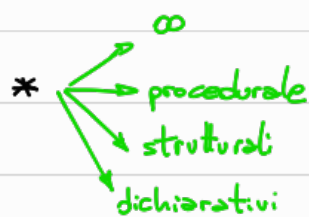
Linguaggi vicini alla macchina fisica → linguaggi macchina

→ assembly

Linguaggio alto livello



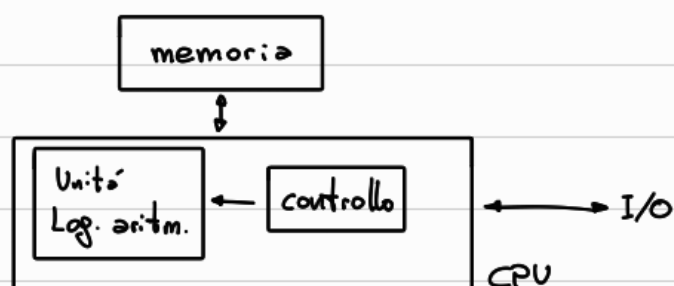
La Variabile come astrazione della cella $v \rightarrow \boxed{\bullet}$



[0:45:00]

Cosa significa implementare un PL

- Architettura della macchina su cui è implementato
- Metodologie di progettazione di programmi



/* Altissimo livello della struttura di una macchina */

PL $\rightarrow$ insieme di costrutti e regole per scrivere programmi

Programma $\rightarrow$ seq. finita di istruzioni del PL che ne rispetta le regole

Obiettivo $\rightarrow$ eseguire il programma

$\Rightarrow$ Macchina astratta per il PL = Data un PL L , M_L (macchina astratta per L) è un insieme di strutture dati e algoritmi che permettono di memorizzare e eseguire programmi scritti in L

macchina intermedia / traduttore

intuitivamente è la macchina il cui linguaggio è esattamente L

$\Rightarrow$ Linguaggio macchina per M = Data una macchina M il linguaggio L_M (linguaggio macchina di M) è il linguaggio che ha come stringhe quelle direttamente interpretabili da M

[1:00:00]



linguaggio macchina L_i utilizzo il linguaggio L_{i-1} e L_i può essere utilizzato ai livelli soprastanti:

Implementare $L \equiv Realizzare M_L$

↳ tradurre L in un linguaggio macchina sottostante **compilazione**

↳ eseguire L sulla macchina sottostante **interpretazione**

$\Rightarrow$ **compilazione** = traduzione esplicita da L a L'

$\Rightarrow$ **interpretazione** = traduzione implicita = simulazione L in L'

$Prog^L \rightarrow$ programmi in L

$D \rightarrow$ insieme dei dati manipolati dai programmi in L

$P^L \rightarrow P^L \in Prog^L \quad P^L : D \rightarrow D$

$\forall input \in D, P^L(input) = output \in D$

INTERPRETE (realizzazione della macchina astratta con soluzione interpretativa - trad. impl.)

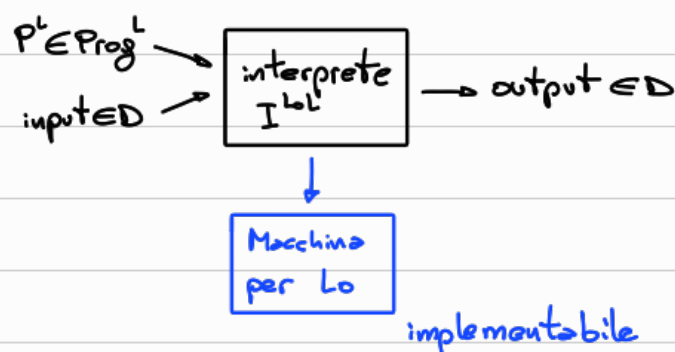
Def: L, P^L : un interprete per L (scritto in L_0)

è un programma $I^{L_0L} : (Prog^L \times D) \rightarrow D$ // MdT universale

$\downarrow$ $\downarrow$ $\downarrow$
 programma input output

$I^{L_0L}(P^L, input) = output$ t.c. $P^L(input) = output$

$I^{L_0L}(P^L, input) = P^L(input)$ **macchina universale**



Struttura

begin

go := true

while go do //potenzialmente infinito

FETCH (opcode, opinfo) at PC;

DECODE (opcode, opinfo);

program counter

if opcode needs arguments then

FETCH(args)

case opcode of

op1 EXECUTE (op1, args);

op2

⋮
HALT : go = false;

if opcode has result then

STORE (result);

PC += size(opcode)



[1:27:00]